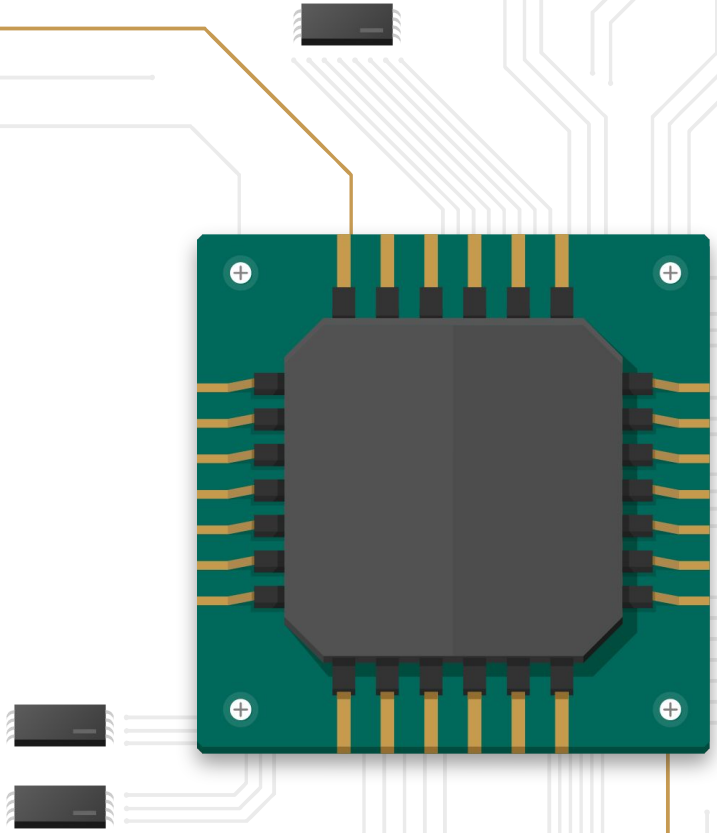
如何打造高吞吐高可用的帳戶餘額系統

從系統分析到系統開發



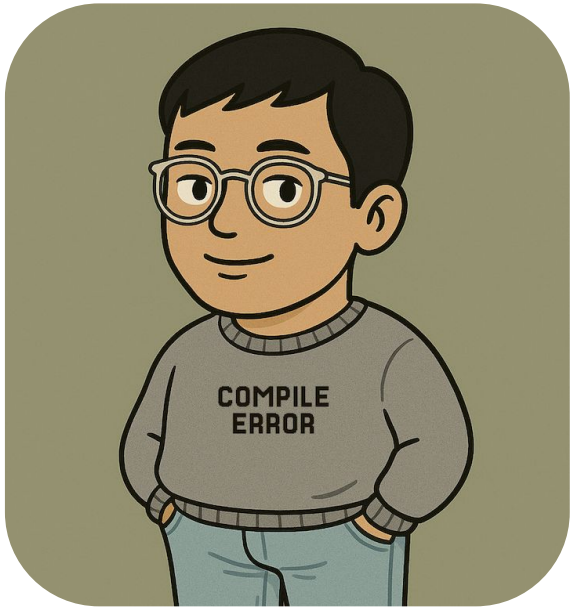
我是誰

Vic

後端工程師

交易所 & 遊戲公司

Thread: chill.vic.22



目錄

01

功能需求
非功能需求

02

如何克服高併發效能
瓶頸

03

如何乘載大量用戶&
實現高可用

04

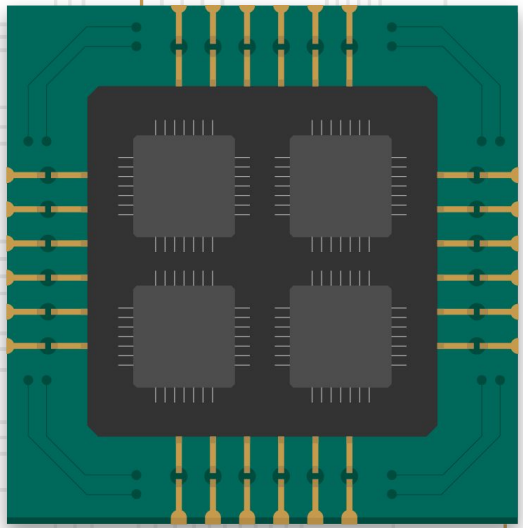
如何實現分散式
Transaction

05

axs 專案實戰

06

Q & A



01

功能需求
非功能需求

什麼是帳戶餘額系統

業務場景

- 銀行支付
- 交易所
- 遊戲

交易所 - 功能性需求

寫入行爲

- 餘額更新
- 不同幣種餘額更新
- 單筆多筆更新

寫入限制

- 冪等性
- 一致性，餘額不能負數
- 驗證請求
- 可追溯

查詢行爲

- 單筆，多筆餘額查詢

查詢限制

- 延遲不能太高

系統整合

- 跨服務之間資料一致性

交易所 - 非功能性需求

系統規模

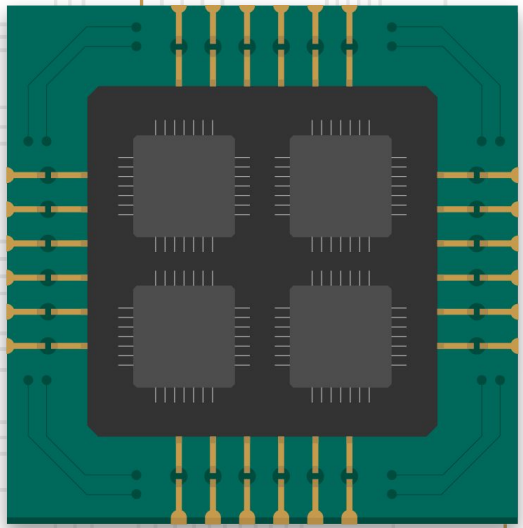
- 千萬用戶
- DAU 50 ~ 100 萬

吞吐量跟延遲

- P95 latency < 50ms
- 100w req/sec

可用性

- Zero downtime upgrade
- Auto failover



02
如何克服高併發效能瓶頸

效能瓶頸是啥？

DB UPDATE

- 樂觀鎖
- P90 Latency 1sec

```
UPDATE user_accounts
SET balance = balance + ?
WHERE user_id = ? AND balance + ? > =0
```

無鎖方案

- 單線程
- 不同用戶不同線程
- 批次更新

```
UPDATE user_accounts ua
INNER JOIN (
    SELECT 1 as user_id, 50.00 as amount UNION ALL
    SELECT 2, 100.00 UNION ALL
    SELECT 3, 75.50
) as temp ON ua.user_id = temp.user_id
SET ua.balance = ua.balance + temp.amount
WHERE ua.balance + temp.amount >= 0;
```

10 w records update cost 100ms
不穩定

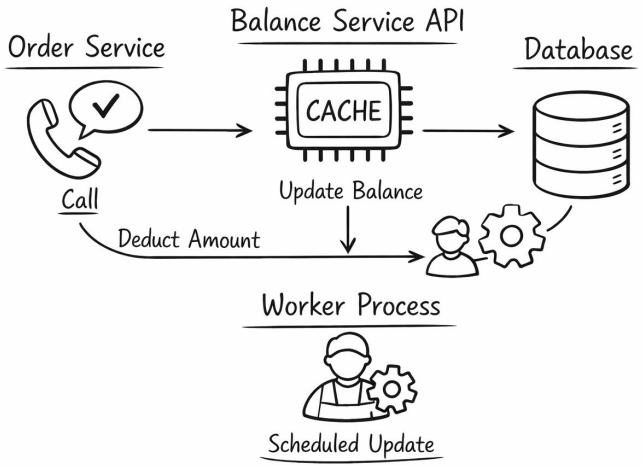
不能用 DB 那是 Redis 嗎？

Redis

- Lua Script 批次更新
- TCP Round Trip
- 不穩定

Application Cache

- 微秒等級操作
- 穩定



Cache 的挑戰

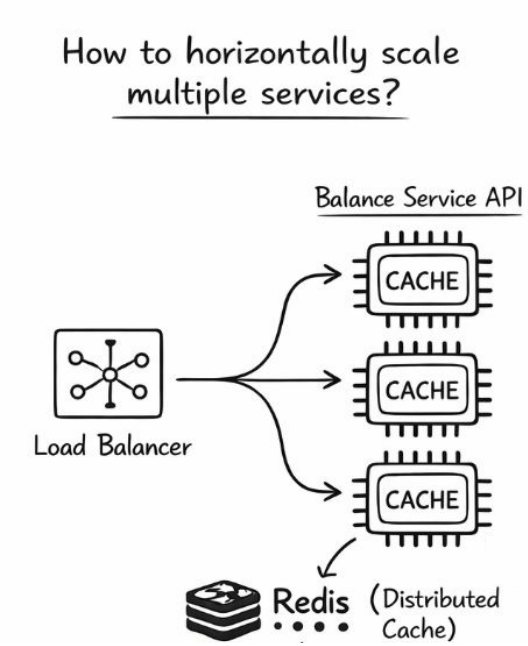
如何 Scale 多台 Server？

如何確保資料可靠，不遺失？

如何 Scale 多台 Server？

多台帳戶餘額系統

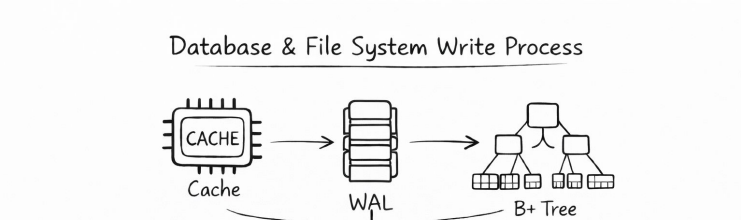
- 單個用戶單線程更新
- Sharding
- Load Balancer / Queue
- Redis



如何確保資料可靠，不遺失？

資料庫如何實現 Durability？

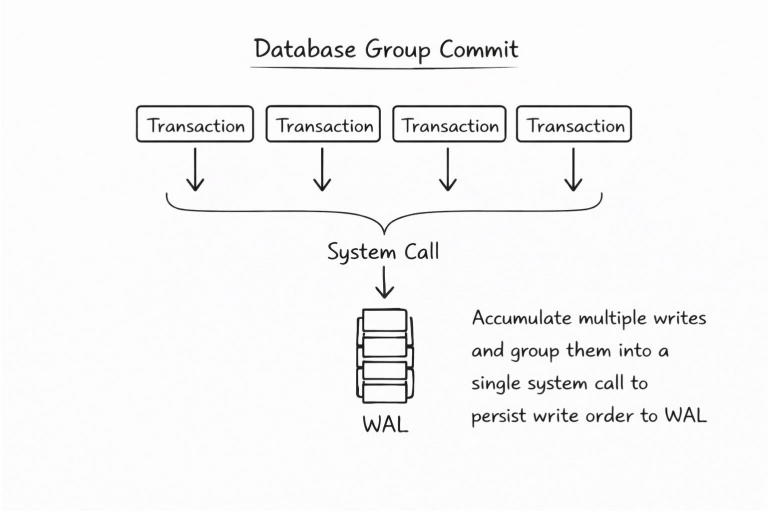
- UPDATE 在記憶體完成
  (e.g buffer pool, shared buffer)
- WAL 順序 I/O 取代隨機 I/O
- Crash 後使用 WAL 重放
- WAL 就是持久化的 Message Queue



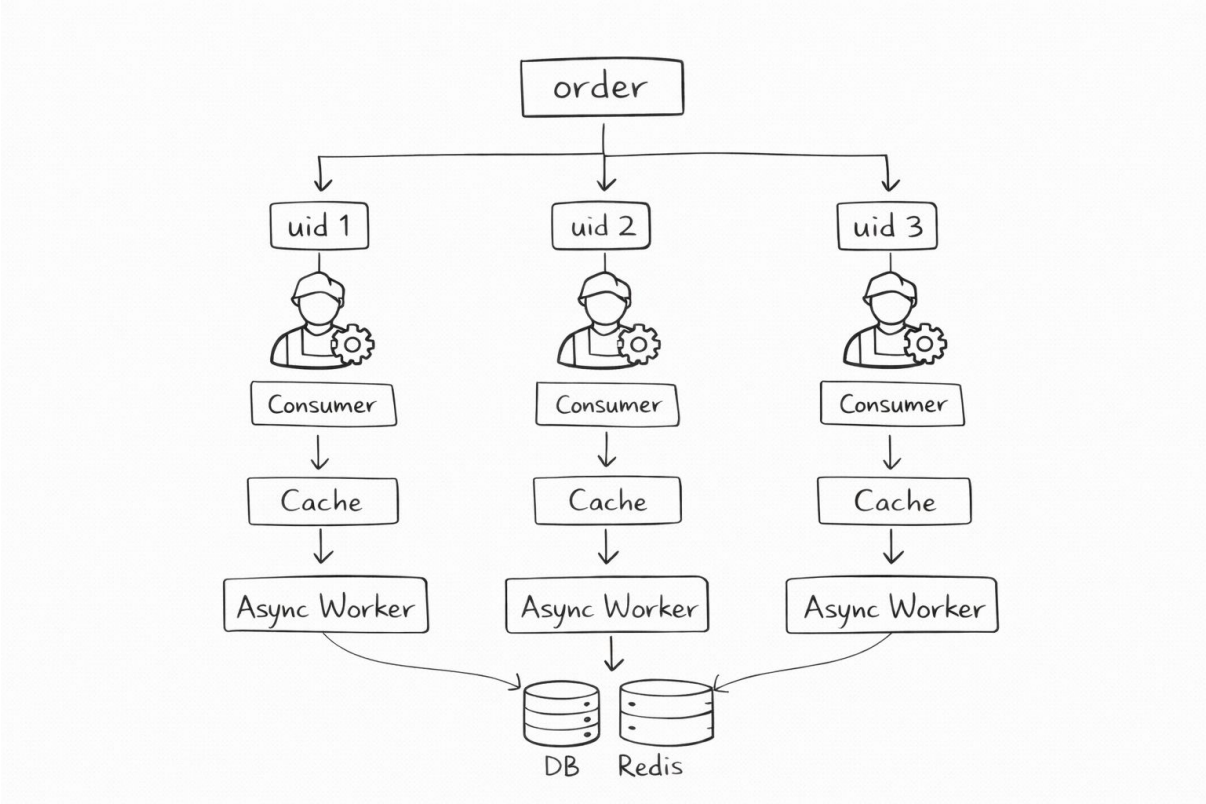
順序寫入的好處

瞬間大量吞吐

- 隨機 I/O 壓垮 Disk
- Group Commit
- 多個 transaction
  一次 system call



用 Message Queue 實現 Scale + Durability



Event Driven & Event Sourcing

Event Driven

- 服務之間通訊模式
- Async 解耦，提供可用性
- 資料最終一致性
- AP 系統

Event Sourcing

- 資料儲存模式
- 儲存可追溯事件
- 審計
- 快照

Event Sourcing 架構選擇

CDC

- Binlog + debezium

好處

- 順序性 At least once
- 無代碼嵌入

缺點

- Producer 邏輯被封裝
- Binlog 讀取延遲

Application Producer

- Outbox Pattern

好處

- Producer 邏輯可控
- 延遲較低

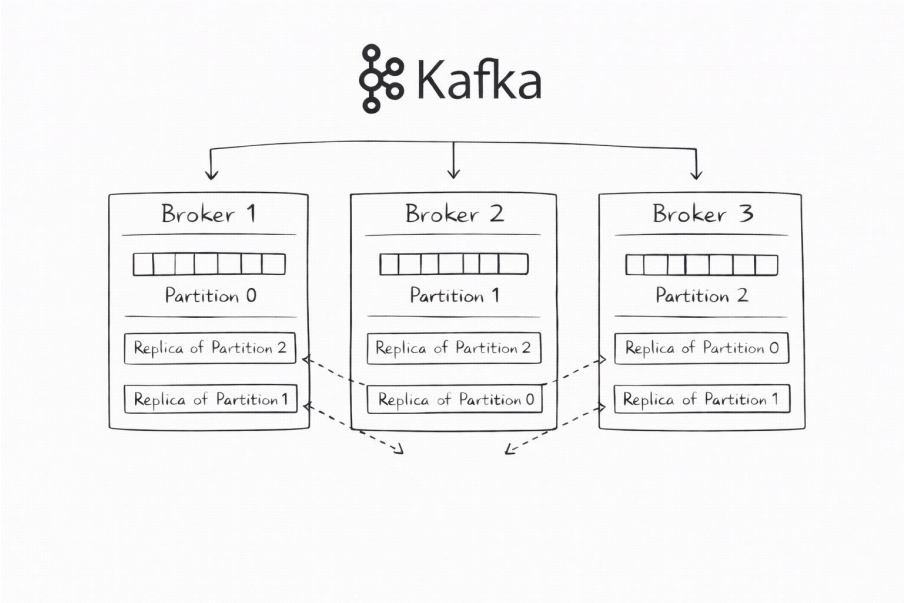
缺點

- 開發複雜
- 無法順序性 At least once

Message Queue 的選擇

Kafka - 高吞吐首選

- Consumer Group
- Scalability



Message Queue 的選擇

Kafka Produce 優化

- log.preallocate fs 加速
- .log 檔案 kernel cache

Kafka Consume 優化

- .index 檔案 mmap
- Sendfile - Zero copy

Kafka 並非低延遲首選

- Batch Producer
- Write System Call
- Pull 模式

Message Queue 的選擇

低延遲首選

- Nats
- Rabbit MQ

MQ 機制

- Push
- In Memory
- Message 路由表

吞吐量低

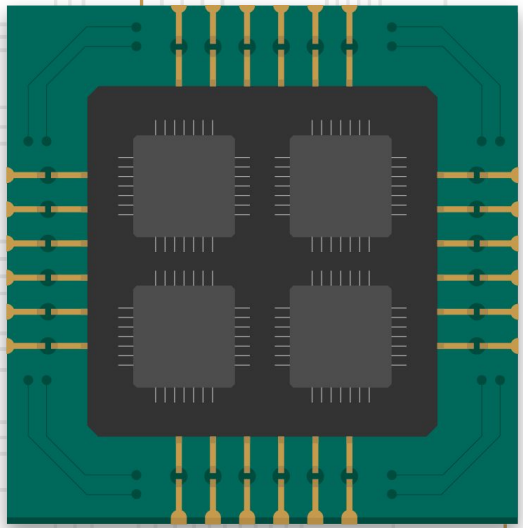
- Produce > Consumer
- 訊息堆積能力差
- Flow Control
- 不可回放

選 Kafka，還需要 outboux pattern嗎

Kafka 可持久化跟回放 (WAL)，還需要 outbox pattern？

Outbox Pattern - 業務層級

- 審計查詢 by user_id or by event_id
- Atomic 操作
- Unique key 實現冪等



03

如何乘載大量用戶＆實現高可用

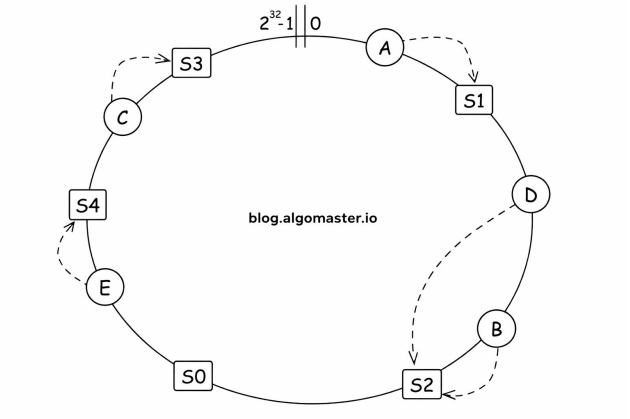
資料分區

單台 DB Table Partition vs 分散式 DB Sharding

- Table Partition => 不同 Partition 不同 B+Tree
- DB Sharding => 水平擴充

Sharding 策略 - Consistent Hashing

- 降低 hot spot
- Resharding 友善



Sharding 欄位：account_id？

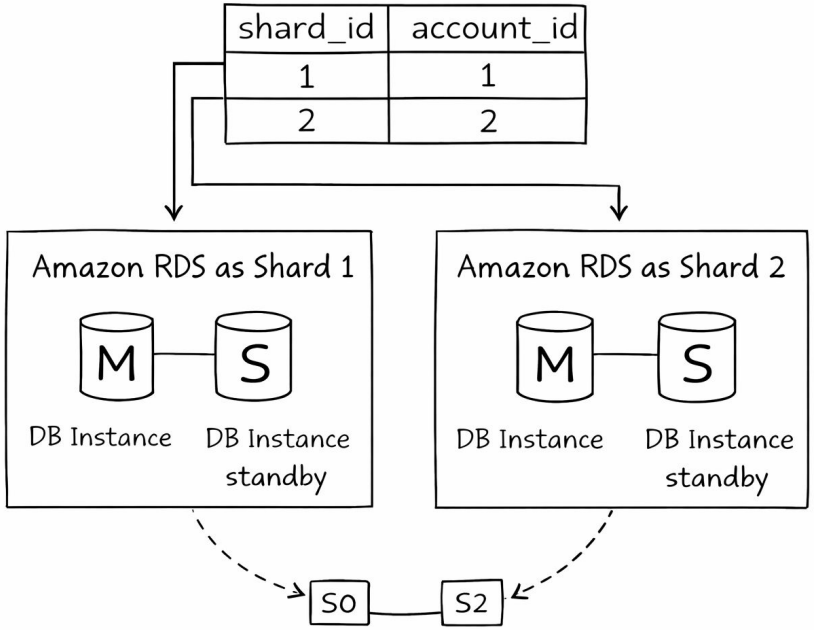
業務邏輯可定義

- 大戶
- 散戶

遷移用戶

- 散戶 => 大戶

自定義 shard_id



分散式資料庫選型

MongoDB

- Coinbase
- Sharding
- MySQL & PostgreSQL？

MySQL

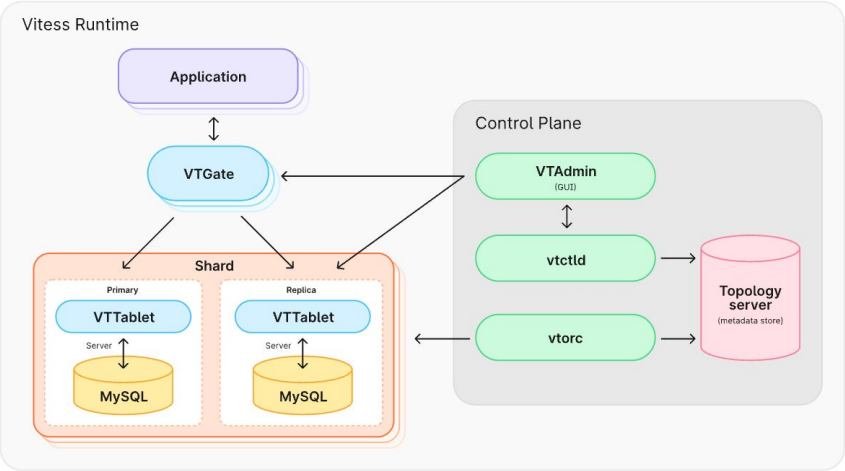
- Vitess (youtube)
- TiDB 不是真的 MySQL

PostgreSQL

- Citus
- PgDog

分散式資料庫選型 - Vitess

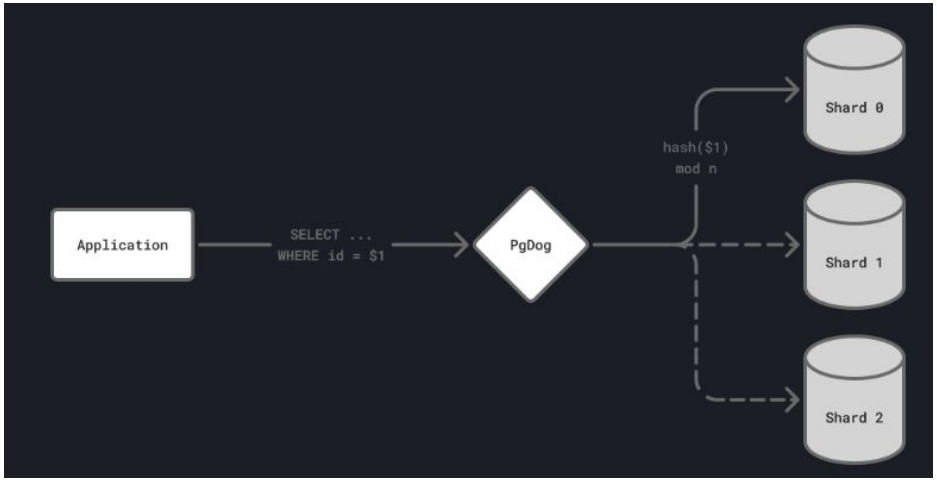
- Sharding routing (zk)
- Re-sharding
- Auto failover
- Sidecar connection pool
- Lookup index & 2FA



*select * from orders join accounts on orders.account_id = accounts.id WHERE orders.created_at BETWEEN ? AND ?*

分散式資料庫選型 - PgDog

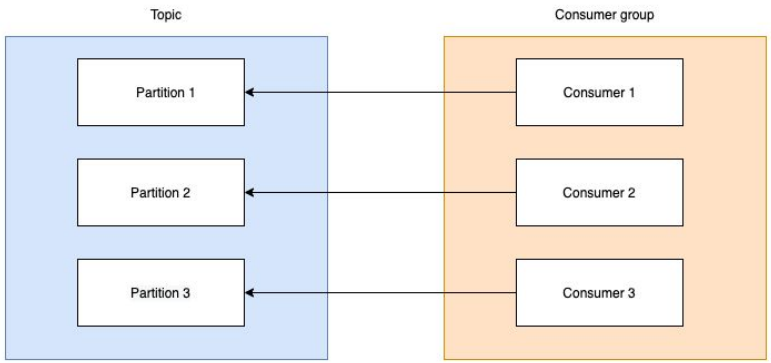
- Sharding routing
- Auto failover
- Toml 管理
- Connection pool
- Rust tokio (epoll)



如何實現 Consumer 高可用？

Kafka Consumer Group

- Rebalance
- 1 ~ 5 sec



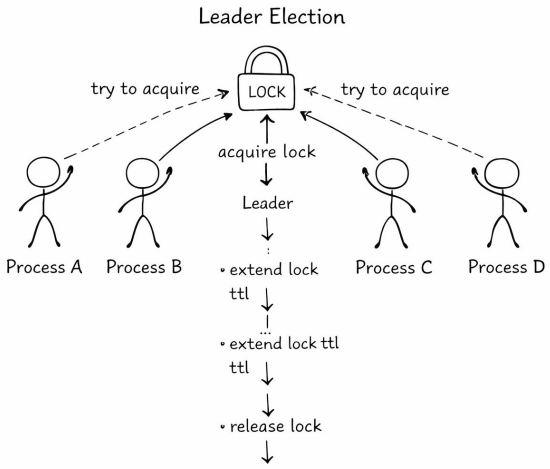
Assign API (指定 partition)

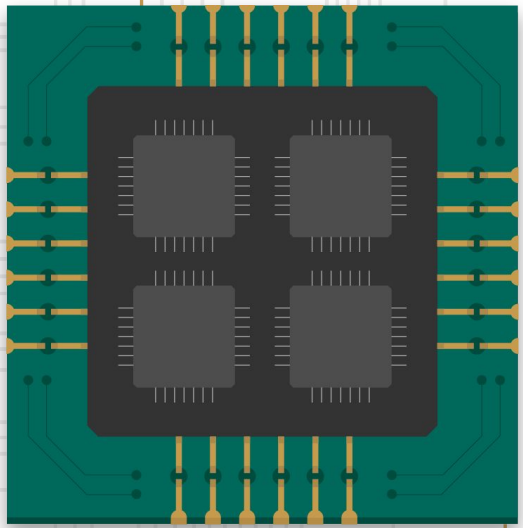
- Group ID 無效
- 多個 Consumer Assign 相同 Partition
- 自行管理 partition & offset

如何實現 Consumer 高可用？

Leader Election

- 多個 Consumer 競爭 Partition Lock
- 搶到為 Leader 執行 Consume
- Shutdown 會 Delete Lock
- Running 會 Extend Lock TTL





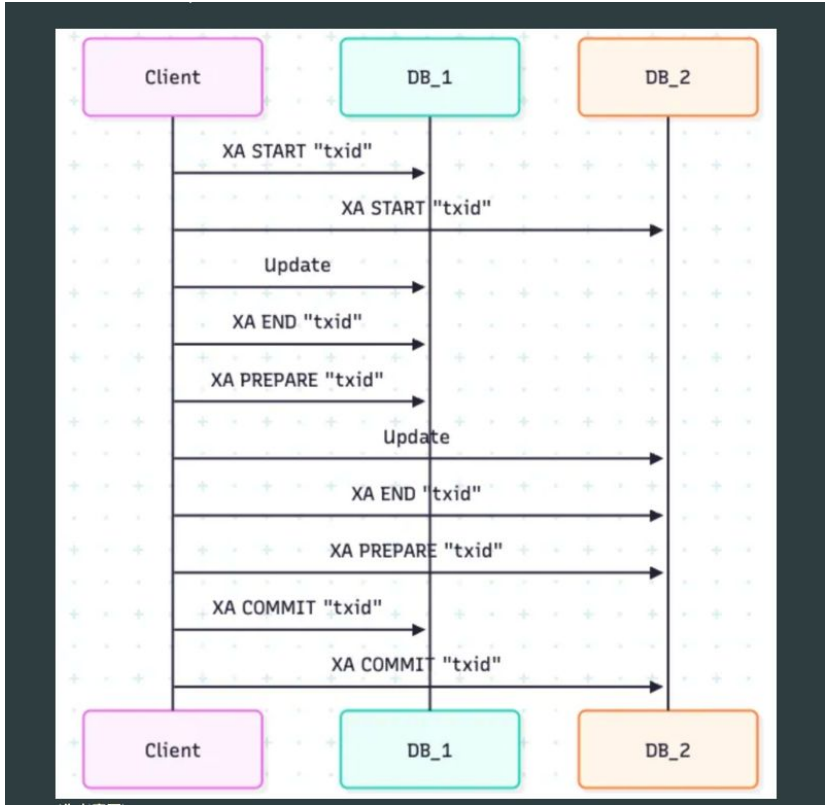
04

如何實現分散式 Transaction

最經典的 Two Phase Commit

XA 轉帳

- 避免 COMMIT 失敗
- 先扣款 => 加錢
- DeadLock
- 網路延遲 Update Lock



業務層級的 Two Phase Commit - TCC

Try -> Confirm / Cancel

- Try
  - Add 100 => temp_balance
  - Move 100 => locak_balance
- Confirm
  - move temp_balnce => real_balance
  - Deduct 100 from lock_balance
- Cancel
  - Rollback Try Operation

優勢

- 每個操作 Transaction 獨立
- 鎖顆粒變小

業務層級的 Two Phase Commit - TCC

網路異常 Confirm 失敗

- Retry
- Tx ID => Master Transaction
- Check Master Transaction Status

Orchestration (編排式)

- 中心化管理者
- Sync API => 資料一致性強
- 低延遲，吞吐量低

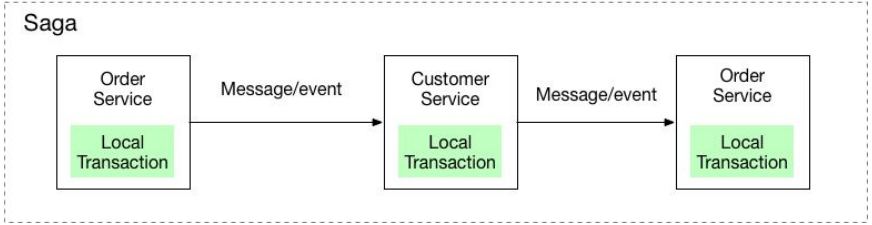
EDA 分散式 Transaction - Saga

Choreography (協同式)

- Transaction 各自執行
- 延遲高 (最終一致)
- 批次處理, 吞吐量大

轉帳

1. Order => Balance A 扣款
2. Balance => Order OK
3. Order => Balance B 加錢
4. Balance => Order OK



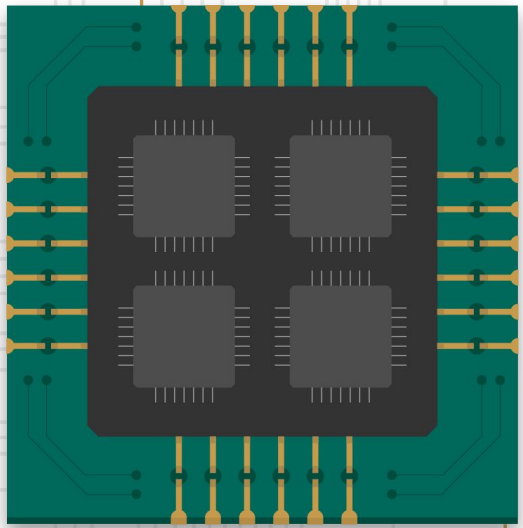
EDA 分散式 Transaction - Saga

沒收到 Result Message 怎麼辦？

- Cron Job : Order => Balance Check Status
- 所有服務與Balance 串接都要 Cron Job?

Balance Server Ack + Callback 機制

- 中心化 Cron Job
- Order 收到 Result Message => Ack API => Balance Server
- 沒有 Ack 的紀錄，Callback 回 Order Service

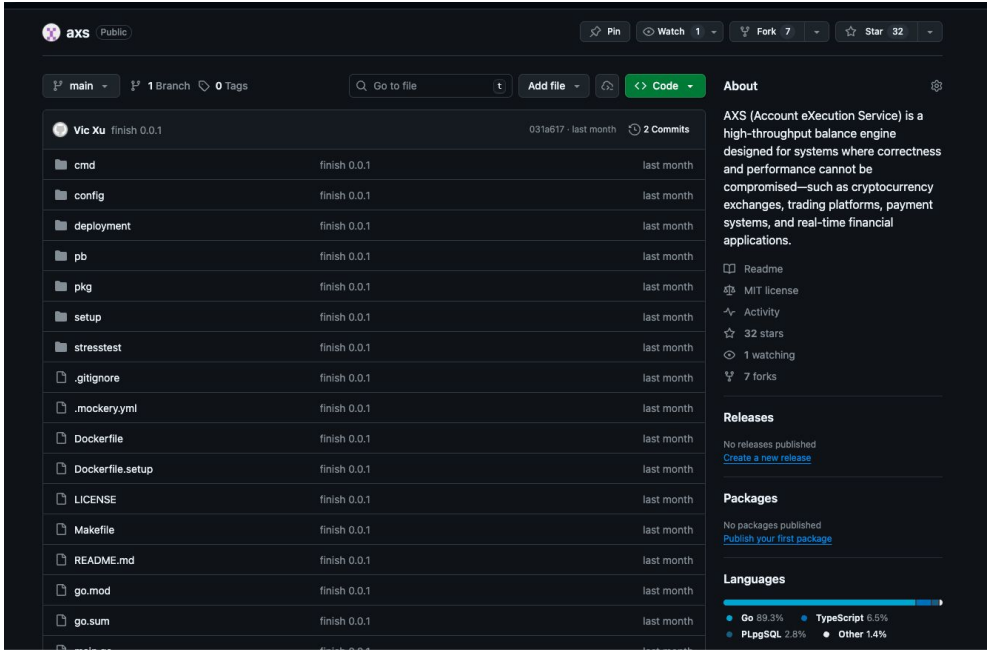


05

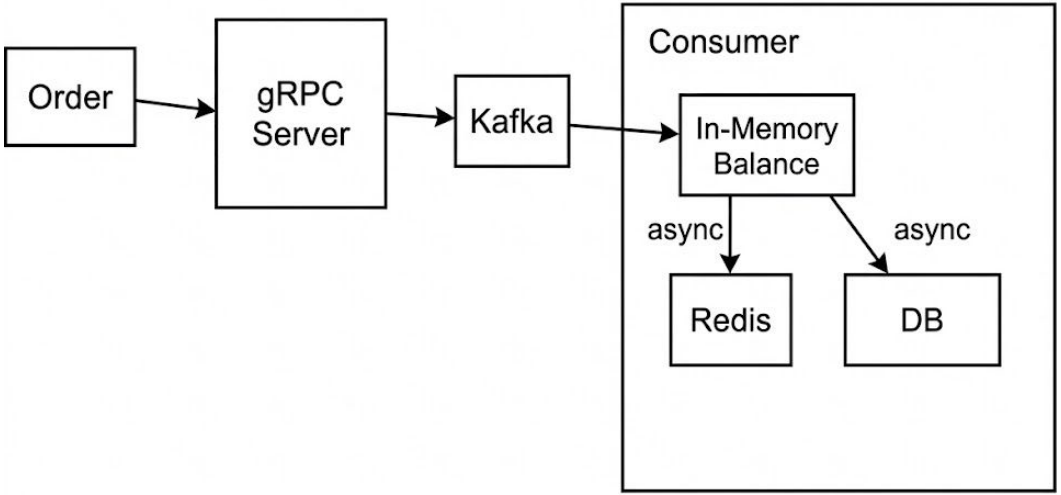
axs 專案實戰

Talk is cheap, show me the code!

1. 架構圖
2. Grpc Server
3. Batch Consumer
4. Cache 套件選擇
5. Redis Worker
6. DB Flush Worker

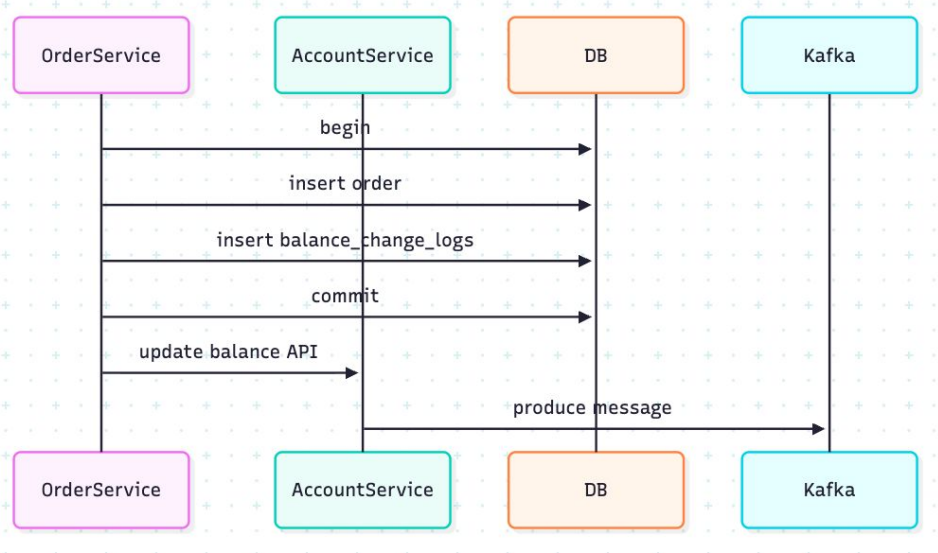


axs - 架構圖


Order
gRPC Server
Kafka
Consumer
In-Memory Balance
async
async
Redis
DB

axs - Grpc Server

- Update Balance API test
  - Insert change_log
  - Producer
- Update Balance API real
  - Producer
- Query Status API
- Query Balance API



axs - Grpc Server

Update Balance API 封裝 Producer

- 確保 Producer Partition Strategy 一致性
- API 層驗證 request sign (HMAC-SHA256)

Insert change_log Unique Key 冪等性

- Unique Key Check
- 無法跨 Sharding / 跨 Partition Unique

axs - Grpc Server

Insert Unique Key 為何變慢？

- 資料庫更新單位是 Page
- Page 有多筆資料同時更新
- Unique Key 多 Lock Index Page
- Page Lock 是 Latch

Latch vs Row Lock

- Latch In Memory Lock
  操作完就釋放
- Row Transaction Lock
  Commit / Rollback 才釋放

axs - Grpc Server

移除 Unique Constraints 如何實現冪等？

- 用 Redis Setnx + TTL
- event_id + timestamp
- Time windows 的冪等
- Timestamp 過期的請求就拒絕

axs - Batch Consumer

Kafka Consumer

- fetch.min.bytes & fetch.wait.max.ms
- Poll API => Message
- Batch Message ?

Batch Message

- Message Buffer Array
- Ticker Thread (goroutine)
- Two Thread => Lock
- 有辦法單線程嗎？

axs - Batch Consumer

Batch Message

- Poll timeout 機制
- Long timeout
- Short timeout

```go
msg, err := c.mqReader.ReadMessage(newCtx, readTimeout)
newCtx = c.setLogger(newCtx, msg)
if err != nil {
    if mq.IsTimeoutError(err) {
        if len(batchMsg) > 0 {
            err := c.handler.ProcessEvents(newCtx, batchMsg)
            if err != nil {
                c.handleErr(newCtx, err, batchMsg)
            }
            logger.GetLogger(newCtx).Debug().Msgf("batch processed due to timeout, batch size: %d", len(batchMsg))
            batchMsg = batchMsg[:0]
        }
        readTimeout = c.maxReadTimeout
        newCtxCancel()
        continue
    }
    logger.GetLogger(newCtx).Error().Err(err).Msg("failed to read message from mq")
    newCtxCancel()
    continue
}
logger.GetLogger(newCtx).Debug().
    Str("topic", msg.GetTopic()).
    Int32("partition", msg.GetPartition()).
    Int64("offset", msg.GetOffset()).Msg("receive message")

readTimeout = c.batchingDelay
batchMsg = append(batchMsg, msg)
if len(batchMsg) >= c.batchSize {
    err := c.handler.ProcessEvents(newCtx, batchMsg)
    if err != nil {
        c.handleErr(newCtx, err, batchMsg)
    }
    batchMsg = batchMsg[:0]
    readTimeout = c.maxReadTimeout
    logger.GetLogger(newCtx).Debug().Msgf("batch processed due to reach batch size, batch size: %d", len(batchMsg))
}
newCtxCancel()
```

axs - Cache 套件選擇

go-cache - 8.8k star

go-generics-cache - 不用 type 轉換

Object-based cache

大量 object GC overhead

GC mark & sweep

Sweep 會 scan heap 中所有物件 => 消耗 CPU

axs - Cache 套件選擇

bigcache - 8.1k star

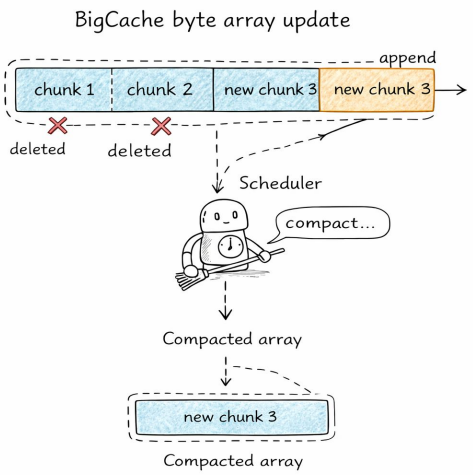
One big bytes array + hash[key]int

Marshal + unmarshal (Batch 優化)

GC 兩個物件 hashmap + bytes array

UPDATE 是 append only

Compact byte array



axs - Redis Worker

Goroutine Pool

餘額 snapshot 併發寫入 redis

Last Write Win 解決 race condition

Redis key {hash tags} 支援 sharding

```lua
while argv_i <= argv_len do
    -- Extract block metadata
    local key = KEYS[key_i]
    key_i = key_i + 1
    local uid       = ARGV[argv_i];
    argv_i = argv_i + 1;
    local shard_id  = ARGV[argv_i];
    argv_i = argv_i + 1;
    local cnt       = tonumber(ARGV[argv_i]);
    argv_i = argv_i + 1;

    -- Write uid and shard_id metadata (always overwrite)
    redis.call("HSET", key, "uid", uid)
    redis.call("HSET", key, "shard_id", shard_id)

    -- Process each currency update in this block
    for _ = 1, cnt do
        local currency     = ARGV[argv_i];
        argv_i = argv_i + 1
        local new_available = ARGV[argv_i];
        argv_i = argv_i + 1
        local new_frozen   = ARGV[argv_i];
        argv_i = argv_i + 1
        local new_epoch    = tonumber(ARGV[argv_i]);
        argv_i = argv_i + 1

        local avail_key = currency .. ":a"
        local frozen_key = currency .. ":f"
        local epoch_key  = currency .. ":epoch"

        -- read old epoch
        local old_epoch_str = redis.call("HGET", key, epoch_key)
        local old_epoch = old_epoch_str and tonumber(old_epoch_str) or nil

        -- LWW: apply update only if epoch is newer
        if old_epoch == nil or new_epoch > old_epoch then
            redis.call("HSET", key, avail_key, new_available)
            redis.call("HSET", key, frozen_key, new_frozen)
            redis.call("HSET", key, epoch_key, new_epoch)
        end
    end
end
```

axs - DB Flush Worker

批次更新如何不落後 In Memory 更新速度？
DB 單執行緒，批次更新 => 1w / 100ms => 10w/s

更新時如何知道資料庫與記憶體資料不同步？
相同用戶併發更新了，記憶體到資料庫同步掉訊息了

Leader Election 如何避免腦裂？
Leader GC 導致 lock expire，兩個 consumer 消費同筆資料

axs - DB Flush Worker

批次更新如何不落後 In Memory 更新速度？

```
UPDATE user_accounts
SET balance = CASE id
    WHEN 1001 THEN balance + 100.50
    WHEN 1002 THEN balance + 250.00
    WHEN 1003 THEN balance - 50.00
    WHEN 1004 THEN balance + 1000.00
    ELSE balance
END
WHERE id IN (1001, 1002, 1003, 1004);
```

SWITCH CASE 更慢

- **Parse 語法更久**
- **AST 更複雜，Optimizer 運行更久**

```
UPDATE user_accounts ua
INNER JOIN (
    SELECT 1 as user_id, 50.00 as amount UNION ALL
    SELECT 2, 100.00 UNION ALL
    SELECT 3, 75.50
) as temp ON ua.user_id = temp.user_id
SET ua.balance = ua.balance + temp.amount
WHERE ua.balance + temp.amount >= 0;
```

優化方向

- **讓語法變簡單**

axs - DB Flush Worker

批次更新如何不落後 In Memory 更新速度？

```
CREATE TEMPORARY TABLE temp_updates
(
    id BIGINT PRIMARY KEY,
    delta_amount DECIMAL(20, 8)
);

INSERT INTO temp_updates (id,
delta_amount) VALUES
(1001, 100.50),
(1002, 250.00),
(1003, -50.00),
(1004, 1000.00),
... -- 可以插入幾萬條
(9999, 500.00);
```

```
UPDATE users u
INNER JOIN temp_updates t ON u.id = t.id
SET u.balance = u.balance + t.delta_amount,
u.updated_at = NOW();
```

axs - DB Flush Worker

批次更新如何不落後 In Memory 更新速度？

LOAD 語法，更適合大量 INSERT

不用解析語法樹

資料傳輸效率更高，不會有 max bytes 限制

```
=== RUN   TestPGDaoSuite
2025/12/09 08:53:17 goose: no migrations to run. current version: 20251122070912
=== RUN   TestPGDaoSuite/TestApplyLargeBalanceChanges
    pg_dao_test.go:78: new ApplyAccountBalanceChanges apply 10k take 154.060917ms
    pg_dao_test.go:86: old ApplyAccountBalanceChanges apply 10k took 592.583875ms
=== RUN   TestPGDaoSuite/TestApplyMediumBalanceChanges
    pg_dao_test.go:99: new ApplyAccountBalanceChanges apply 2k took 57.963458ms
    pg_dao_test.go:107: old ApplyAccountBalanceChanges apply 2k took 126.327458ms
=== RUN   TestPGDaoSuite/TestApplySmallBalanceChanges
    pg_dao_test.go:120: new ApplyAccountBalanceChanges apply 300 took 8.408292ms
    pg_dao_test.go:128: old ApplyAccountBalanceChanges apply 300 took 26.260333ms
```

```
mysql.RegisterReaderHandler("data", func()
io.Reader {
    return createCSVBuffer(trades)
})
defer mysql.DeregisterReaderHandler("data")

db.Exec(`
    LOAD DATA LOCAL INFILE 'Reader::data'
    INTO TABLE temp_updates
    FIELDS TERMINATED BY ','
    LINES TERMINATED BY '\n'
    (account_id, currency,  delta_amount)
`)
```

axs - DB Flush Worker

更新時如何知道資料庫與記憶體資料不同步？

資料一致性

- **確保 amount >= 0**
- **Version 確保 in memory & db 資料同步**
- **Return row_affected**

UPDATE SQL

- **新幣種要改用 INSERT**
- **CTE 提升可讀性**

MySQL **優化**

- MySQL UPDATE Not Found => GapLock
- **SELECT 時標記 not found**

```
batchUpdateAccountBalanceSQL = `
        WITH try_update AS (
            UPDATE account_balances ab
            SET available = ab.available + t.available_delta,
                frozen    = ab.frozen + t.frozen_delta,
                updated_msec = t.updated_msec,
                version= t.next_version
            FROM temp_balance_write_records t
            WHERE ab.account_id = t.account_id
            AND ab.shard_id = t.shard_id
            AND ab.currency_code = t.currency_code
            AND ab.available + t.available_delta >= 0
            AND ab.frozen + t.frozen_delta >= 0,
            AND ab.version = t.current_version
            RETURNING t.account_id, t.shard_id, t.currency_code
        ),
        missing AS (
            SELECT t.*
            FROM temp_balance_write_records t
            LEFT JOIN try_update u
            ON u.account_id = t.account_id
            AND u.shard_id   = t.shard_id
            AND u.currency_code = t.currency_code
            WHERE u.account_id IS NULL
        ),
        try_insert AS (
            INSERT INTO account_balances (
                account_id, shard_id, currency_code,
                user_id, available, frozen, updated_msec
            )
            SELECT
                m.account_id, m.shard_id, m.currency_code,
                m.user_id, m.available_delta, m.frozen_delta, m.updated_msec
            FROM missing m WHERE m.available_delta >= 0 AND m.frozen_delta >= 0
            RETURNING account_id
        ),
        validate AS (
            SELECT
                (SELECT count(*) FROM try_update) AS updated_rows,
                (SELECT count(*) FROM try_insert) AS inserted_rows
        )
        SELECT *
        FROM validate;
```

axs - DB Flush Worker

Leader Election 如何避免腦裂？

Leader Lock 設計

- DB Table
- 跟 UPDATE Balance 放在同個 transaction
- Extend leader lock TTL
- 檢查 fencing_token

```sql
CREATE TABLE partition_leader_locks (
    topic TEXT NOT NULL,
    partition INTEGER NOT NULL,
    commit_offset BIGINT NOT NULL,
    updated_msec BIGINT NOT NULL DEFAULT
    updater_svc_id TEXT NOT NULL,
    leader_svc_id TEXT,
    lease_expired_msec BIGINT NOT NULL,
    fencing_token BIGINT NOT NULL,
    PRIMARY KEY (topic, partition)
);
```

```
sql := `UPDATE ` + PartitionLeaderLocksTable + `
        SET commit_offset = $1,
            updated_msec = $2,
            updater_svc_id = $3
        WHERE topic = $4 AND partition = $5 AND fencing_token = $6
    `
```

axs - DB Flush Worker

最終的 Flush Worker Transaction

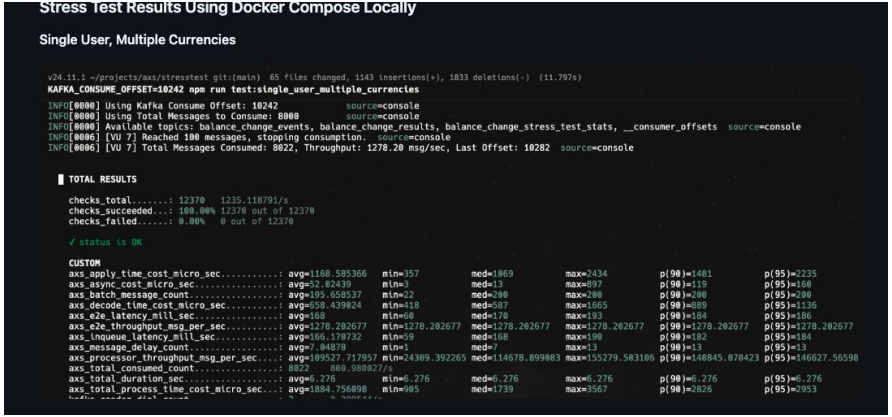
1. UPDATE leader lock => 避免腦裂
2. UPDATE change_logs status => 冪等
3. UPDATE balance => 確認資料一致

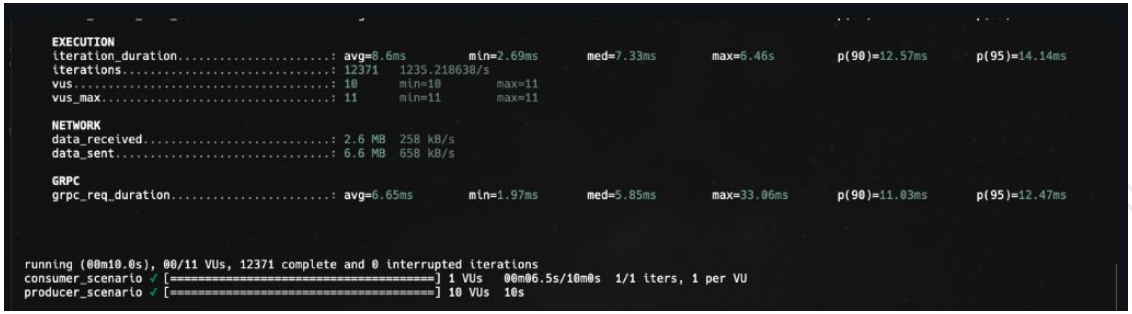
```
77
78  batchUpdateBalanceChangeLogStatusSQL = `
79      UPDATE balance_change_logs bcl
80      SET status = t.status,
81          reject_reason = t.reject_reason,
82          updated_msec = t.updated_msec
83      FROM temp_balance_change_log_status t
84      WHERE bcl.idempotency_key = t.idempotency_key
85      AND bcl.change_id = t.change_id AND bcl.status = 0;
86  `
```

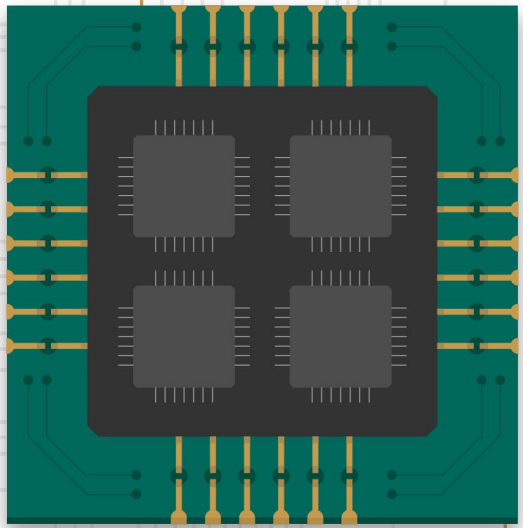
axs - 壓測

K6 consumer 壓測

- K6 打 API
- K6 consumer 消費 metrics
- Consumer 單線程 avg 10w msg/sec
- Grpc P95 latency 12msec





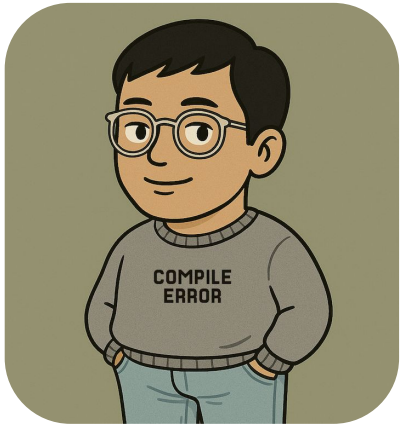


05

Q & A

Chill Vic

資料庫/程式語言

DevOp/作業系統

系統設計